

The Movin backend

What each component is, how the components work together during a ride, a payment or a sign-in, and how to rebuild any part of it, or the whole server, from scratch.

DATE

16 September 2026

STATUS OF THE SERVICE

Mauritania live · Algeria built, not yet open

SOURCE REPOSITORY

nammayatri-algeria/nammayatri, branch
algeria/osrm-routing

RUNS ON

One VPS, Docker Compose, about 20 containers

MAIN REFERENCE IN THE REPOSITORY

Backend/dev/local-stack/README.md

SCOPE

The ride backend and its public edge. The mobile app and the website each have their own repository and are covered only where they connect.

01 Summary

The Movin backend is built on **Namma Yatri**, an open-source ride-hailing platform from India, written in Haskell. We run a fixed 2023 version of it, adapted to Mauritania and Algeria. Everything around it (maps, SMS codes, payments, push notifications, photos) is our own, small, and replaceable.

~20 containers on one server	2 countries in one deployment	0 paid map, routing or search APIs	44 min to rebuild the backend binaries from cold (8 warm)
--	---	--	---

Three ideas explain the whole design

1.

Endpoints are data, not code.

The backend finds Google, SMS or Firebase through URLs stored in its database. We point those URLs at our own services, so most integrations need no rebuild.

2.

Change around the backend before changing inside it.

The order of preference is database config, then a small Node.js service in front, then a patch to the Haskell code.

3.

One server, fully described in the repository.

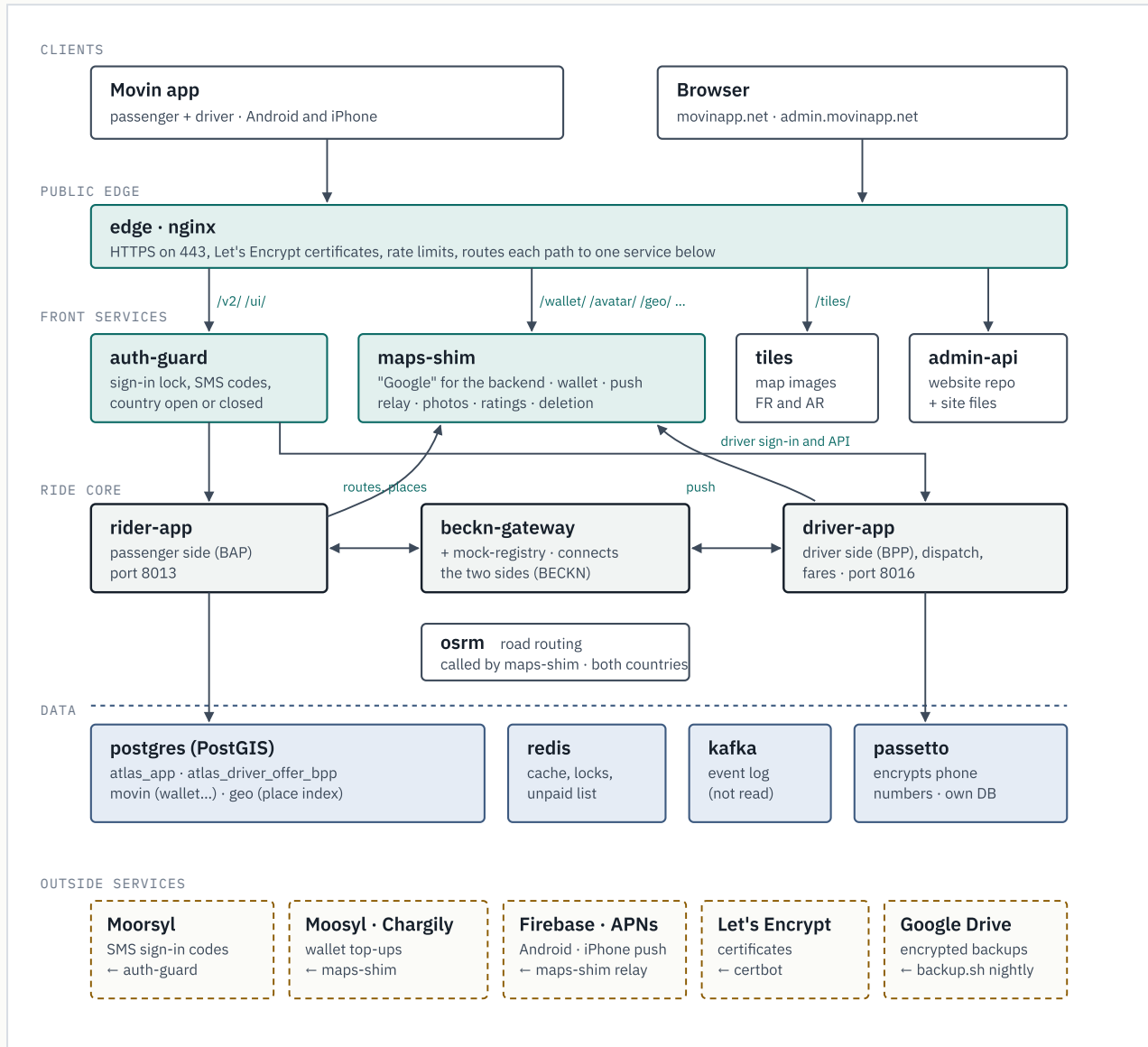
Every container, script and setting is in Backend/dev/local-stack, so the server can be rebuilt from git, a backup, and a short list of secret keys.

Where things are

PIECE	WHERE IT LIVES	WHAT IT IS
Ride backend	<code>nammayatri-algeria/nammayatri</code>	This report. Haskell services plus our Node.js services, nginx and scripts.
Mobile app	<code>.../namma-yatri-frontend</code>	React Native (Expo). One app for passengers and drivers, Android and iPhone.
Website and admin	<code>.../Movin_DZ_Website</code>	Public site, admin console and admin-api. Deployed on the same server, reads the same database.
Public addresses	<code>api.movinapp.net</code>	The API for both apps. <code>movinapp.net</code> and <code>admin.movinapp.net</code> for the website.

02 The architecture in one picture

Only three ports are open to the internet: 22 for SSH, 80 for certificate renewal and redirects, and 443 for HTTPS. Everything else listens on the server's own loopback address and is reached through the **edge**, which decides which requests go where.



Teal boxes are Movin's own code. Grey boxes are the Namma Yatri binaries. Blue boxes are data stores. Dashed boxes are outside companies. The website's admin-api also reads the Postgres database. Its arrow is left out to keep the picture readable.

03 Each component

Grouped by role. For each one: what it does, what it is made of, and what breaks if it stops.

A • The public edge

edge

ny-edge · nginx:alpine

[OURS](#) · [CONFIG](#)

The only public entrance. It serves HTTPS for `api.movinapp.net` and the website names. It sends `/v2/` (passengers) and `/ui/` (drivers) through the auth-guard, sends our own paths (`/wallet/`, `/avatar/`, `/geo/`, `/tiles/...`) to their service, and refuses everything else. It reloads itself every 6 hours so renewed certificates are picked up.

Why it matters If it stops, both apps and the website are offline. A new service path is unreachable from phones until the edge has a `location` block for it.

certbot

ny-certbot

[OFF THE SHELF](#)

Renews the Let's Encrypt certificates twice a day when they are within 30 days of expiry.

Why it matters If it stops, the certificates expire after about 90 days and phones refuse to connect.

auth-guard

ny-auth-guard · Node.js · :8031

[OURS](#) · [CODE](#)

Sits in front of both Haskell apps. Its jobs:

- **Limits code guesses:** 3 attempts, then the number is locked for 15 minutes.
- **Handles SMS codes:** it sends the code through Moorsyl and checks what the user typed. The backend only knows a fixed code, so the guard substitutes it once the real code is correct.
- **Opens or closes countries:** `OPEN_COUNTRIES`, today Mauritania only.
- **Blocks unpaid drivers:** a driver with an empty wallet gets `WALLET_EMPTY`.
- **Personal codes:** approved drivers have their own code as a fallback when SMS fails.

Why it matters This is the lock on sign-in. Without it anyone could try all 10,000 codes in about ten minutes. It must be running before port 443 is opened.

B • The ride core: Namma Yatri binaries

All five run from **one Docker image** (`ghcr.io/nammayatri-algeria/ny-backend`, tagged locally `ny-rider:patched`), each with a different program. They use host networking because their 2023 configuration expects every service at `localhost`.

rider-app

ny-rider · Haskell · :8013

[UPSTREAM](#) · [PATCHED](#)

The passenger side, called the BAP in the BECKN protocol. It handles passenger accounts, ride search, estimates and offers, booking, the ride's progress, and ratings. It stores data in the `atlas_app` schema and runs its own database migrations at startup.

Why it matters If it stops, passengers can't sign in or request a ride.

driver-app

ny-driver · Haskell · :8016

UPSTREAM · PATCHED

The driver side, called the BPP. It handles driver accounts, positions, going online, **dispatch** (which drivers are offered a request), fares, offers, starting a ride with the 4-digit code, and ending it. Schema `atlas_driver_offer_bpp`. There is **one driver "merchant" per country**, each with its own tariff and service area.

`Why it matters` If it stops, a search finds no drivers and no price. The server also spends most of its idle processor time here.

beckn-gateway + mock-registry

ny-beckn-gateway :8015 ·
ny-mock-registry :8020

UPSTREAM · BUILT BY US

The two sides never call each other directly for a search. The rider-app sends `/search` to the gateway, which looks up every driver-side participant in the registry and forwards the request to each. The registry holds each party's public key, which is how signed messages are checked.

`Why it matters` If either stops, a search returns a route but no price, and drivers never see the request.

mock-google

ny-mock-google · :8019

UPSTREAM · FALLBACK

Upstream's fake Google Maps. The maps-shim answers everything it can itself and passes the rest here.

`Why it matters` Rarely used, but an unknown maps call without it fails with `GOOGLE_MAPS_API_ERROR`.

C · Maps, without Google

maps-shim

ny-maps-shim · Node.js · :8030

OURS · CODE

Two roles in one small service.

1. The backend's "Google". The backend's Google Maps URL (stored in its database) points here:

- /directions is answered by OSRM.
- /place/autocomplete, /place/details and /geocode are answered from our own place index in Postgres: 111,000+ places, with reviewed Arabic names.

2. The Movin features the 2023 backend doesn't have, one file each:

wallet.js	Driver wallet: top-ups through Moosyl (MRU) or Chargily (DA), a ledger, and the day charge taken at the first ride.
restricted.js	Publishes the list of unpaid drivers to a Redis key that the patched dispatch reads.
push-relay.js	The backend's Firebase URL points here. Android tokens are forwarded to Firebase. iPhone tokens go to Apple (APNs) with the app's own French/Arabic/English text.
avatars.js	Profile photos, stored in the avatars volume.
rating.js · fleet.js	A person's own rating, and the nearby cars shown on the passenger's map.
deletion.js	Account deletion requests, refused during a ride.
geo.js	/geo/country: which country the sign-in screen shows, based on the IP address.
subscription.js	The old monthly subscription, replaced by the wallet on 7 Sep. Due for removal.

Why it matters If it stops, no routes, no place search, no push notifications, no wallet and no photos. It is the most important piece of Movin's own code.

osrm

ny-osrm · :5000

OFF THE SHELF

Road routing (distance, duration and route shape) on a graph built from OpenStreetMap for Mauritania and Algeria together. About 3 ms per route.

Why it matters No routes means no price estimate. The graph is built once by a script, not by the container.

tiles

ny-tiles · tileserver-gl · :8035

OFF THE SHELF · OUR STYLES

Draws the map shown under the route, from the same OpenStreetMap data. It has two styles: French (basic-preview) and Arabic (movin-ar), with a font pack that includes Arabic letters.

Why it matters If it stops, the app's map is blank, but rides still work.

D • Data

postgres

ny-postgres · PostGIS 15 ·
volume ny-pgdata

CRITICAL

One database, atlas_dev, with several schemas:

- atlas_app (passengers) and atlas_driver_offer_bpp (drivers): owned by the Haskell migrations. Never write to them by hand except through reviewed SQL scripts.
- movin: our tables (wallet, ledger, deletion requests).
- geo: the place index, which can be rebuilt.
- atlas_registry: the registry's data.

Service areas, tariffs and service URLs are all rows here.

Why it matters This is the business. It is backed up every night, except the place index, which can be rebuilt.

passetto + passetto-db

ny-passetto · ny-passetto-db

CRITICAL

An encryption service. Phone numbers are stored encrypted in the main database. The keys are in this separate small database.

Why it matters If it's down, every sign-in fails with a server error. Without the passetto keys, a restored database has unreadable phone numbers, which is why the backup includes both.

redis

ny-redis

IMPORTANT

Short-lived data: cached configuration, search locks, sign-in counters, and the movin:unpaid list that dispatch reads.

Why it matters Trap: the merchant configuration is **cached** here. After changing it in the database, the matching Redis keys must be deleted or the old value stays in use.

kafka

ny-kafka · loopback only

REQUIRED, UNUSED

The Haskell services write events to it. Nothing of ours reads them today, but the services expect it to be there.

Why it matters Its connection warnings in the logs are harmless.

E • Helpers

CONTAINER	PURPOSE	NEEDED IN PRODUCTION?
proxy · driver-proxy · osrm-proxy	Re-expose host-networked ports (8014, 8017, 5001) for access through an SSH tunnel.	No. Convenience only.
map	Service-area demo page on :8025.	No
adminer	Database browser on :8026, reachable only through a tunnel.	No
movin-admin-api	Website back office (Fastify, :8040). Lives in the website repository, deployed into the same compose file.	Yes, for the admin console

04 How they work together

Five everyday journeys, step by step. They show which components each action depends on, which is also the order to check them in when something fails.

Sign-in (passenger or driver)

1. The app sends the phone number to `api.movinapp.net/v2/auth` (drivers: `/ui/auth`). The **edge** passes it to the **auth-guard**.
2. The guard checks the country is open and the number isn't locked, then asks **Moorsyl** to text a code.
3. The user types the code. The guard checks it with Moorsyl, then sends the backend its fixed internal code.
4. **rider-app** or **driver-app** finds or creates the account. The number is encrypted through **passetto**, and a session token comes back to the app.

A ride, from request to drop-off

1. **Search.** The passenger picks a destination: the app searches places through the **maps-shim** (the Postgres place index). The request goes to **rider-app**, which asks the maps-shim for the route, and the maps-shim asks **OSRM**.
2. **Across the protocol.** rider-app sends the search to the **beckn-gateway**. The gateway finds the driver side in the **registry** and forwards the request to **driver-app**.
3. **Dispatch.** driver-app takes a per-merchant lock (in **Redis**) and picks drivers with a recent position within the radius, starting at 5 km. It removes unpaid drivers (the Redis list) and any driver not on the passenger's shortlist, computes fares from the country's tariff, and sends the request to the chosen drivers.
4. **Offers.** Drivers accept, and their offers (with the car description) travel back to rider-app. The passenger chooses one and confirms.
5. **Pickup and ride.** The driver's app sends its position every few seconds (stored in Postgres). At pickup the driver enters the passenger's 4-digit code, and at the end the fare is final.
6. **After the ride.** Each side rates the other. The first ride of the day takes the driver's day charge from the wallet.

One measured search (price, route and pinging drivers) costs about 280 ms and a little over one processor core-second. Ride requests, not storage, are what limit the server's capacity.

Driver wallet top-up

1. The driver app calls `/wallet/topup`. The **maps-shim** checks the driver's session with driver-app.
2. It creates a checkout with **Moosyl** (Mauritania) or **Chargily** (Algeria), and the driver pays there.
3. On return, the shim **asks the payment company** for the status. A webhook alone never grants credit.
4. The ledger in `movin` is updated. **restricted.js** refreshes the unpaid list, and the driver can work again.

Push notification

1. An event happens, such as a new offer, driver arrived or ride cancelled.
2. **rider-app** or **driver-app** sends it to the "Firebase" URL in its database, which actually points to the **maps-shim push relay**.
3. Android token: forwarded unchanged to **Firebase**.
4. iPhone token: rewritten with the app's own words and sent to **Apple APNs** with Movin's key.

Admin console

1. The browser opens `admin.movinapp.net`. The **edge** serves the console files and passes `/admin/*` to **admin-api**.
2. `admin-api` uses its own session cookies and at most 5 database connections. It **reads** the ride schemas and **writes** only to the `movin` schema, through reviewed SQL.

05 What we changed from upstream, and where

Every change sits at one of three levels. The higher the level, the cheaper and safer the change.

LEVEL	EXAMPLES	COST OF A CHANGE
1 • Database configuration NO REBUILD	Service areas (both countries, twice: passenger side and driver side), tariffs per country, maps URL pointing to the maps-shim, OSRM, the Firebase URL pointing to the push relay, the answer window for drivers, cancellation reasons.	One SQL script, then clear the Redis cache. Minutes.
2 • Services in front NO REBUILD	auth-guard (SMS, locks, countries), maps-shim (maps, wallet, push, photos...), nginx rules.	Edit JavaScript, copy it to the server, restart one container. Minutes.
3 • Patches to the Haskell code REBUILD	54 patch sites in <code>.github/scripts/algeria/apply-patches.py</code> , listed below.	A CI build (44 min cold, 8 warm), then an image swap and a full ride test.

The Haskell patches, grouped by feature

FEATURE	SITES	WHAT IT DOES
Phone numbers for two countries	7	Accept +222 with 8 digits and +213 with 10 digits instead of India's +91.
Negative coordinates	1	The 2023 code couldn't read a longitude west of Greenwich, so every Mauritanian search failed silently.
Search lock per country	1	With two driver merchants in one process, the second country's search was dropped.
Car shown on each offer	10	Make, model, colour and plate travel with the driver's offer to the passenger.
Passenger picks the drivers	17	A shortlist chosen by the passenger limits who receives the request.
Drivers rate passengers	16	A new route and a stored rating for each passenger.
No top-up, no work	2	Dispatch never offers a job to a driver on the unpaid list.

Important: the build does **not** compile our branch. It downloads upstream Namma Yatri at the fixed commit 03a7531 (March 2023) and applies `apply-patches.py` on top. A Haskell change reaches production **only** by being added to that file. Editing the source tree changes nothing.

Why a 2023 version

Current upstream Namma Yatri can't be run by an outside company. It starts with an empty database and pulls its configuration from Namma Yatri's own private systems. The March 2023 commit is the last one that works on its own with a seeded merchant. Upstream has changed a lot since then, so the source tree and the running binaries disagree in places. The

rule on this project: **to know what the server does, ask the server** (its `/openapi` route list, or the binaries themselves), not the source code.

06 How to rebuild

From the smallest change to the whole server. Each procedure below uses scripts that already exist in Backend/dev/local-stack.

6.1 • Rebuild the Haskell binaries

Needed only for a level-3 change. The build runs for free on GitHub Actions (`.github/workflows/algeria-backend-build.yml`).

01 Add the change to `apply-patches.py`, and test it locally first.

Extract the 2023 commit into a temporary folder and run the script twice: every patch must apply once, then report "already in place". This catches a broken patch in seconds instead of 40 minutes into a build.

02 Start the build

```
git push upstream algeria/osrm-routing:algeria/build-backend --force
```

CI downloads the 2023 source, applies the patches, and compiles `rider-app`, `dynamic-offer-driver-app`, `beckn-gateway` and `mock-registry` inside the fixed build image `fpco/stack-build:lts-16.31` (GHC 8.8.4, Ubuntu 18.04). Don't upgrade that image: binaries built against a newer system library don't start on the server.

03 Wait for "publish image"

About 44 min from cold, 8 min with the cache. The workflow publishes `ghcr.io/nammayatri-algeria/ny-backend:latest` plus a numbered tag. Check the workflow named *algeria: build backend*, not the first green line in the list. Nothing is deployed automatically.

04 Deploy on the server

```
cd /opt/ny/local-stack
bash deploy-backend.sh           # pulls, checks the patch is in both binaries,
tags a rollback, swaps
bash deploy-backend.sh rollback # if anything is wrong: back to the previous
image
```

If there's a database change, apply the SQL **before**

swapping images. Columns are always added as nullable, so the old binary keeps working and rollback stays a simple image swap.

05 Prove a full ride

`probe-booking-flow.py` runs a whole ride from both sides. `probe-two-country-rides.py` checks both countries.

6.2 • Change a Movin service (maps-shim, auth-guard, nginx)

01 Edit and test locally

These are plain Node.js files, mounted into their containers, so there is no image to build.

02 Copy the files to the server and restart that container

```
docker restart ny-maps-shim ny-auth-guard      # or: bash deploy-shims.sh
docker exec ny-edge nginx -t && docker exec ny-edge nginx -s reload
```

Copy files with `scp`, which keeps the mounted file in place. Replacing a mounted file with `tar` leaves the container serving the old copy. A new public path also needs a `location` block in the

deployed

`edge/nginx.conf`, which contains the website blocks that the repository copy doesn't.

6.3 • Change configuration (tariffs, areas, URLs)

01 Run the matching script

For example `apply-tariff.sh`, `apply-fcm.sh`, `apply-search-window.sh`, `mauritania-geofences.sql`. Every tariff or merchant statement names its `merchant_id`. An unkeyed statement changes the other country too.

02 Clear the cache

Delete the Redis keys for `Merchant` and `TransporterConfig`. A restart alone does not clear them.

6.4 • Refresh the maps (routing, map images, place search)

```
bash maps-two-countries.sh build      # new OSRM graph + tiles from fresh OpenStreetMap data,
beside the live ones
bash maps-two-countries.sh switch     # serve them, run checks, roll back automatically if a
check fails
bash tiles-arabic.sh                  # re-apply the Arabic style afterwards
```

Never run `geocoder-prepare.sh` load on the live server. It drops the place index, including the reviewed Arabic names. Add places with `geocoder/append-country.sql`.

6.5 • Rebuild the whole server from nothing

For the day the server is lost. The pieces are all in git, the nightly backup and the list of secret keys below.

This full sequence has **not yet been rehearsed end to end** on a fresh machine. Each step is a script that has run in production, but the chain as a whole is untested. A rehearsal on a temporary VPS is recommended (section 8).

01 New server

Ubuntu 24.04, at least 6 cores, 12 GB RAM and 100 GB of disk (the current server is Contabo, 6 vCPU, 12 GB, 193 GB). Install Docker. Firewall: allow only 22, 80 and 443 (`ufw limit` on SSH). Create the `ny` user and `/opt/ny`.

02 Code

Clone the repository and copy `Backend/dev/local-stack` to `/opt/ny/local-stack`. Write `.env` with `BIND_ADDR=127.0.0.1` and `MAP_COUNTRY=algeria-mauritania`.

03 Secrets

Put the keys from the list below in `/opt/ny/secrets/` and the `maps-shim` volume. They are never in git.

04 Backend image

`docker pull ghcr.io/nammayatri-algeria/ny-backend:latest`, tagged `ny-rider:patched`. No compiling needed.

05 Bring the stack up

`./setup.sh` fetches the pinned 2023 configuration, creates the database, the registry and `passetto`, and runs a sign-in check.

06 Restore the data

`./backup.sh restore <file>` with the backup downloaded from Google Drive and its passphrase. It restores both the main database and the `passetto` keys, then checks row counts against the manifest in the archive.

07 Maps

`maps-two-countries.sh all`, then `tiles-arabic.sh`. This rebuilds OSRM, the tiles and the place index from OpenStreetMap.

08 Configuration

Re-apply what isn't in the database backup: `apply-search-window.sh` (a file-based setting), `apply-fcm.sh`, and the nightly backup timer (`backup.sh install`).

09 Public edge

Point the DNS for `api.`, `admin.`, `www.` and `movinapp.net` to the new IP. Issue certificates with `certbot`, then start `auth-guard` before edge.

10 Website

From the website repository: `docker/ship.sh`, then add the `admin-api` service to the deployed compose file (see the note below).

11 Prove it

`setup.sh` `verify`, `probe-booking-flow.py`, `probe-push.py`, `probe-wallet-screens.py`, then one real sign-in and one real ride from a phone.

Secrets needed (names only; values are kept outside git)

SECRET	USED BY	IF LOST
Backup passphrase	<code>backup.sh</code>	UNRECOVERABLE Backups can't be opened.
Moorsyl API key	<code>auth-guard</code>	Nobody can sign in by SMS. Get a new key from the Moorsyl console.
Moosyl and Chargily secret keys	<code>maps-shim wallet</code>	No top-ups. Get new keys from each console.
Firebase service account	<code>backend database (via apply-fcm.sh)</code>	No Android push. Create a new key in Firebase.
Apple APNs key (.p8)	<code>maps-shim push relay</code>	No iPhone push. Create a new key in the Apple Developer account.
Driver personal codes file	<code>auth-guard</code>	Drivers fall back to SMS codes. Can be regenerated.
Android signing key	<code>app releases (not the server)</code>	UNRECOVERABLE No updates to the Play Store app.

Never copy the repository's `docker-compose.yml` over the deployed one. The deployed file also contains the website's `admin-api`, and running `--remove-orphans` after such a copy deletes the website API.

07 Running it day to day

Backups

- Every night (systemd timer): the ride schemas plus the pasetto keys.
- Encrypted with AES-256 before leaving the server, then copied to Google Drive with `rclone`.
- About 7 MB, because the 155 MB place index is left out on purpose (it can be rebuilt).
- The script refuses to run if a new schema appears that it doesn't know, so nothing is silently left out.
- `backup.sh restore` tests a backup against a scratch database, never the live one.

Capacity (measured 27 Aug)

OPERATION	MEDIAN	≈ PER SECOND
Ride search	284 ms	4.5
Driver position	37 ms	106
Place search	23 ms	289
Map tile (vector)	8 ms	610
OSRM route	2.6 ms	1459

These are single-request costs extrapolated to a full machine, measured with 36 drivers. The search cost grows with the number of drivers, and no load test has

Security

- Only ports 22, 80 and 443 are reachable. Databases, Redis, Kafka and the admin tools listen on loopback only.
- SSH by key only, rate-limited: the 6th connection within 30 seconds is refused.
- The driver binary's 41 office routes (/dashboard/) are not published.
- No secret is in git.

been run. Storage is not a limit: one completed ride takes 24 kB.

Daily checks

- `https://api.movinapp.net/healthz` answers 200.
- The newest backup is less than 24 hours old (`backup.sh list`).
- Driver positions are fresh. Stale positions make every search return no estimate, with no error anywhere.

08 Risks and recommendations

What would hurt most, in order, and what to do about each.

LEVEL	RISK	RECOMMENDATION
HIGH	One server, no standby. On 31 Aug the provider had an outage that made the server unreachable for about 18 hours.	Rehearse section 6.5 on a temporary VPS and time it. That turns "we could rebuild" into "we rebuild in X hours". Then decide whether a warm standby is worth its monthly cost.
HIGH	Frozen 2023 base. No security updates come from upstream, and the build tools are old (GHC 8.8, Ubuntu 18.04 image).	Keep the backend unreachable except through the edge and the guard (as today). Keep patches few, and add features in the Node.js services where possible. Plan a longer-term decision: stay on this base, or move to a maintained one.
MEDIUM	Test accounts that skip SMS are listed in the auth-guard. The Algerian test accounts must go before Algeria opens.	Empty the list before real launch in each country, as part of a launch checklist.
MEDIUM	Payment keys. The wallet only works if each country's payment key is the live one, not the test one.	Before each country opens, confirm the live key is installed with one small real top-up.
MEDIUM	Dispatch runs one batch only. The allocator service that sends later waves isn't deployed, so the drivers reached on the first attempt are the only ones asked.	Acceptable at pilot scale. If a later allocator is added, it must be built with the same patches first, or it would ignore the wallet gate and the passenger's shortlist.
LOW	Logs. driver-app logs every Redis call at DEBUG level. That costs processor time and keeps only about 4 hours of history.	Lower the log level in its configuration. Free capacity, and more useful history.
LOW	Repository and server drift. The deployed nginx and compose files contain website blocks that the repository copies don't.	Commit those blocks to the repository so a rebuild from git is complete.
LOW	Leftovers from the monthly subscription (<code>subscription.js</code> and its tables).	Remove them. This is already planned in the code restructuring.

Prepared from the repository (Backend/dev/local-stack, `.github/workflows/algeria-backend-build.yml`, `.github/scripts/algeria/apply-patches.py`) and from measurements recorded there. Figures are as measured on the dates given. No passwords or keys appear in this document.